

ProteinMPNN — Helper Guide

A friendly companion to the assignment. Concepts, setup, a starter skeleton, a flags glossary, and a troubleshooting table. Read this alongside the assignment PDF.

How to use this: The **Assignment** PDF tells you *what* to deliver and the exact contracts. This **Helper** explains the *concepts* and gives you a starting point plus a code skeleton. If anything here conflicts with the assignment, the assignment's Appendix A is authoritative.

1. The concepts you need before you start

What is a "backbone"?

A protein is a chain of amino acids folded into a 3D shape. The **backbone** is just the 3D path of that chain — the "wire sculpture" — *without* deciding which amino acid sits at each position. A PDB file stores these 3D coordinates.

What does ProteinMPNN actually do? (inverse folding)

Normal folding: *sequence* → *shape*. ProteinMPNN does the **reverse**: *shape* → *sequence*. You give it a backbone and it answers "**which amino-acid sequence fits this shape?**" It can propose many candidate sequences, and it can also **score** a sequence you already have against the backbone.

What is a "tool" in an AI agent?

Our agent is an LLM that can call Python functions. Each function = a **tool**. The LLM reads a short description, decides to call it, passes arguments, and gets back a **JSON result** it explains to the user. Your two functions become two new tools. That's why your return value must be a clean, JSON-friendly dictionary — the LLM reads it.

What is the "score"? (lower = better)

ProteinMPNN's score is a **negative log-likelihood**: how surprised the model is by a sequence given the backbone. **Lower = the model likes it more = more compatible**. Always say this in your docstrings so the agent explains it correctly.

What is "sampling temperature"?

It controls how varied the output is. **Low (0.1)** gives conservative, high-probability sequences close to the model's top choices; **higher (0.2–0.3)** gives more varied ones. Default to 0.1.

2. The workflow you're building

user uploads / fetches a PDB backbone



```
proteinmpnn_tools.py  (YOUR FILE)

design_sequences(pdb, chains, n, temp)
  1. _check_deps()      → torch present?
  2. _find_pdb()        → locate the file
  3. subprocess.run(protein_mpnn_run.py)
  4. _parse_mpnn_fasta(seqs/*.fa)
  5. return {"success": True, ...}

score_sequence(pdb, sequence)
  same idea, --score_only mode
```

↓ clean JSON dict

mentor registers it → the LLM agent calls it

3. Getting started, step by step

1. **Clone & install** (see assignment A.1). Verify torch works:

```
python -c "import torch; print(torch.__version__)"
```

2. **Run the design example by hand** on the public structure [8EK5](#) (download it via assignment A.2), designing the scFv binder chain [E](#). **Expected:** a new file `mpnn_out/seqs/8EK5.fa` containing a native line plus 4 designs.
3. **Open the .fa and copy a real sample** into a scratch file — you'll use it to build and test your parser *before* wiring up subprocess.
4. **Build the parser first** (pure Python, no torch). Then the subprocess call. Then assemble the result dict. This order means you always have something testable.

Suggested order: parser → unit test → subprocess wrapper → result dict → smoke test. Don't write the whole file then debug it all at once.

4. Starter skeleton (fill in the TODOs)

This is a scaffold to get you moving — it is **not** complete. Match the real conventions in `aibinder_tools.py` as you go.

```

"""
proteinmpnn_tools.py – Agent-callable wrappers for ProteinMPNN
(protein sequence design & scoring from a 3D backbone).
"""

import os, re, json, tempfile, subprocess, shutil
from pathlib import Path
from typing import Any, Dict, List

# ---- locate ProteinMPNN (no hardcoding) -----
PROTEINMPNN_DIR = os.environ.get("PROTEINMPNN_DIR", "/opt/ProteinMPNN")
RUN_SCRIPT = os.path.join(PROTEINMPNN_DIR, "protein_mpnn_run.py")

# ---- lazy dependency check (copy the aibinder_tools pattern) -----
_TORCH_OK = False
try:
    import torch          # noqa
    _TORCH_OK = True
except ImportError:
    pass

def _check_deps() -> Dict[str, Any]:
    """Return an error dict if something is missing, else None."""
    missing = []
    if not _TORCH_OK:
        missing.append("torch")
    if not os.path.isfile(RUN_SCRIPT):
        missing.append(f"ProteinMPNN (set PROTEINMPNN_DIR; looked in {PROTEINMPNN_DIR})")
    if missing:
        return {"success": False,
                "error": "Missing dependencies: " + ", ".join(missing) +
                        ". Install torch and clone ProteinMPNN."}
    return None

# reuse the existing PDB resolver so users can pass just a filename
from aibinder_tools import _find_pdb    # noqa

# =====
# Parser – write this FIRST and unit-test it (no torch needed)
# =====
def _parse_mpnn_fasta(text: str) -> List[Dict[str, Any]]:
    """
    Parse ProteinMPNN's output FASTA into a list of records:
    [{"header": {...}, "sequence": "..."}, ...]
    The first record is the native sequence. See assignment Appendix C.
    HINT: split into >header / sequence pairs, then split the header on
    commas, then each piece on the first '=' to get key=value pairs.
    """
    records = []
    blocks = [b for b in text.strip().split(">") if b.strip()]
    for b in blocks:
        lines = b.splitlines()
        header_line = lines[0]
        seq = "".join(lines[1:]).strip()
        meta: Dict[str, Any] = {}

```

```

    for piece in header_line.split(","):
        if "=" in piece:
            k, v = piece.split("=", 1)
            meta[k.strip()] = v.strip()
    records.append({"header": meta, "sequence": seq})
return records
# TODO: convert score / global_score / seq_recovery to float,
# TODO: handle the very first record's name (it has no '=' before the first comma).

```

```

# =====
# Milestone 2 – design
# =====
def proteinmpnn_design_sequences(pdb_path: str, chains_to_design: str = "",
    num_sequences: int = 8, sampling_temp: float = 0.1,
    seed: int = 37, model_name: str = "v_48_020") -> Dict[str, Any]:
    dep = _check_deps()
    if dep: return dep
    resolved = _find_pdb(pdb_path)
    if not resolved:
        return {"success": False, "error": f"PDB file not found: {pdb_path}"}

    out_dir = tempfile.mkdtemp(prefix="mpnn_")
    try:
        cmd = ["python", RUN_SCRIPT,
            "--pdb_path", resolved,
            "--out_folder", out_dir,
            "--num_seq_per_target", str(num_sequences),
            "--sampling_temp", str(sampling_temp),
            "--seed", str(seed),
            "--batch_size", "1"]
        if chains_to_design.strip():
            cmd += ["--pdb_path_chains", chains_to_design.strip()]
        # TODO: add --model_name if you expose it

        proc = subprocess.run(cmd, capture_output=True, text=True, timeout=300)
        if proc.returncode != 0:
            return {"success": False,
                "error": f"ProteinMPNN failed: {proc.stderr[-800:]}"}

        stem = Path(resolved).stem
        fa = Path(out_dir) / "seqs" / f"{stem}.fa"
        records = _parse_mpnn_fasta(fa.read_text())
        # TODO: build the result dict EXACTLY as in assignment Appendix A
        #     (native_sequence, native_score, sequences[], etc.)
        return {"success": True, "pdb_file": resolved, "_raw_records": records}
    except subprocess.TimeoutExpired:
        return {"success": False, "error": "ProteinMPNN timed out (300s)."}
    finally:
        shutil.rmtree(out_dir, ignore_errors=True) # or keep for debugging

# =====
# Milestone 3 – score-only (TODO: implement with --score_only 1)
# =====

```

```
def proteinmpnn_score_sequence(pdb_path: str, sequence: str = "",
                               fasta_path: str = "", chains_to_score: str = "", seed: int = 37) -> Dict[str, Any]:
    ...    # TODO

if __name__ == "__main__":
    r = proteinmpnn_design_sequences("8EK5.pdb",
                                     chains_to_design="E", num_sequences=2)
    print(json.dumps(r, indent=2)[:1500])
```

5. ProteinMPNN flags glossary (the ones you'll use)

Flag	What it does
<code>--pdb_path</code>	Path to a single input PDB structure (quick mode, no folder parsing).
<code>--pdb_path_chains</code>	Which chain(s) to design , space-separated, e.g. <code>"A B"</code> . Omit = all chains.
<code>--out_folder</code>	Where outputs go. Sequences appear under <code><out>/seqs/<stem>.fa</code> .
<code>--num_seq_per_target</code>	How many candidate sequences to generate.
<code>--sampling_temp</code>	Diversity dial as a string : <code>"0.1"</code> safe ... <code>"0.3"</code> diverse.
<code>--seed</code>	Random seed — set it so runs are reproducible.
<code>--batch_size</code>	Keep at <code>1</code> for CPU/testing.
<code>--score_only 1</code>	Score mode: don't design, just score the sequence(s).
<code>--path_to_fasta</code>	FASTA of external sequence(s) to score (with <code>--score_only</code>).
<code>--model_name</code>	Which weights, e.g. <code>v_48_020</code> (default). Leave default unless asked.
<code>--fixed_positions_jsonl</code> (<i>stretch</i>)	Lock specific residues so they don't change. Built via helper scripts.
<code>--unconditional_probs_only</code> (<i>stretch</i>)	Output per-position amino-acid probabilities instead of designs.

6. Troubleshooting

Symptom	Likely cause & fix
<code>ModuleNotFoundError: torch</code>	Wrong conda env, or torch not installed. <code>conda activate proteinmpnn</code> then <code>pip install torch</code> .
<code>FileNotFoundError: protein_mpnn_run.py</code>	<code>PROTEINMPNN_DIR</code> not set / wrong. Point it at your cloned repo: <code>export PROTEINMPNN_DIR=/path/to/ProteinMPNN</code> .
Output <code>.fa</code> not found	It's under <code><out_folder>/seqs/<stem>.fa</code> , not directly in <code>out_folder</code> . Use the PDB <i>stem</i> (filename without <code>.pdb</code>).
Subprocess returns non-zero	Read <code>proc.stderr</code> — usually a bad chain ID or a malformed PDB. Print the full command you built and run it by hand.
Parser gives wrong scores	The first record's name has no <code>=</code> before the first comma — handle it specially. Convert numeric strings to <code>float</code> .
<code>json.dumps</code> raises "not serializable"	A numpy value sneaked in. Wrap numbers with <code>float()</code> / <code>int()</code> .
Hangs forever	Always pass <code>timeout=</code> to <code>subprocess.run</code> and catch <code>TimeoutExpired</code> .
Wrong chain designed	<code>--pdb_path_chains</code> selects chains to redesign ; everything else is fixed context. Double-check chain letters in the PDB.

7. When (and how) to ask your mentor

Asking good questions is a skill. Before pinging your mentor, try for ~20 minutes, then ask with context:

- **Good:** "Running design on 8EK5, chain E — the `.fa` has the native line but only 1 design instead of 4. Here's my command and the stderr tail. Any idea why?"
- **Less useful:** "It doesn't work."

Always include: the exact command/code, what you expected, what happened, and the error text. Check in at each **milestone** (after Part A, after Milestone 2, after Milestone 3).

8. Suggested 3-week plan

Week	Focus	Done when...
1	Read the summary + paper README; Part A manual runs; <code>ERIC_notes.md</code> .	You can design & score by hand and explain the <code>.fa</code> format.
2	Parser + unit test; Milestone 2 design function; smoke test.	<code>proteinmpnn_design_sequences</code> returns the Appendix A dict on a <code>Test_pdb</code> file.
3	Milestone 3 score function; polish, docstrings, error cases; open PR.	All "Definition of Done" boxes checked; PR submitted.

A final note. ProteinMPNN is a command-line program — most of the job is running it cleanly and turning its text output into a structured dictionary. Work in small, testable steps, and check in with your mentors at each milestone rather than saving questions to the end.